CONTENTS

Advanced | AI system architecture | Condensed | IBM Technology

Generative vs Agentic AI: One Call versus a Loop

Understand what actually changes when a model stops answering and starts acting.

For engineers deciding whether a feature needs a model call or an agent · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

Watch the source video | Read the full lesson | Read the highlights

BEFORE YOU START

- How a chat completion API call works: messages in, message out
- Comfort reading Python
- Some exposure to function/tool calling, though the lesson explains it

---

Overview

The difference between generative and agentic AI is not a difference in model. It is a difference in control flow. A generative system is one request and one response: you prompt, it produces, it stops. An agentic system takes that same model and puts it inside a loop that observes, decides, acts on the world, and feeds the result back in.

That structural change is what produces every other difference. A single call is cheap, fast, bounded and easy to evaluate. A loop is none of those things — it can run for minutes, call external systems, cost an unpredictable amount, and fail in ways a single call cannot. In exchange it can pursue a goal you specified once, across many steps, without you supervising each one.

Both rest on the same foundation. A large language model provides the reasoning in both cases; in the agentic case it is doing something narrower and more specific than people usually assume — it chooses the next action and interprets the last result. Everything else in an agent is ordinary software.

KEY TAKEAWAYS

- ✓ Generative and agentic AI differ in control flow, not in the model — one is a call, the other is a loop containing that call.
- ✓ A generative system is reactive: it waits for a prompt, produces content, and stops.
- ✓ An agentic system is proactive: given a goal, it perceives, decides, acts, and learns from the result, repeating with minimal human input.
- ✓ The same LLM serves as the reasoning engine in both; in an agent it decides the next action and interprets the last observation.
- ✓ Chain-of-thought decomposition is how an agent turns a vague goal into an ordered sequence of executable steps.
- ✓ Generative workflows keep a human curating every output; agentic workflows spend human attention on the goal and the guard rails instead.
- ✓ A loop has no natural stopping point, so termination conditions, step budgets, and failure handling are your responsibility, not the model's.
- ✓ Cost and latency become unbounded and data-dependent the moment you move from one call to a loop.

01 Reactive generation versus proactive goal pursuit

0:00

> One produces content and stops. The other pursues an objective across many actions. The gap between them is a while loop.

KEY POINTS

- → Generative systems are reactive: prompt in, content out, done.
- → Agentic systems are proactive: goal in, sequence of actions out, done when the goal is met.
- → The model is identical; the control flow around it is what differs.
- → An agent's prompt is an objective that outlives any single response.
- → Every practical difference follows from the loop, not from the model.

The same model, two shapes of program

This is the whole distinction in fourteen lines. The top is a function call. The bottom is a control structure that happens to contain one.

```
# generative: one call, one result, done
answer = model.generate(prompt)


# agentic: the call is now inside a loop that can act on the world
state = {"goal": goal, "history": []}

while not done(state):
    observation = perceive(state)          # look at the world
    action      = model.decide(state, observation)   # the LLM's job
    result      = execute(action)          # actually do it
    state       = update(state, action, result)      # learn from it
```

02 What a generative model is actually doing

0:40

> Sophisticated pattern completion: statistical relationships learned from a large corpus, used to predict what plausibly comes next.

KEY POINTS

- → Generation is next-token prediction: a distribution over the vocabulary, sampled at each position.
- → Coherence comes from conditioning on the whole prompt at every step, not from a stored plan.
- → Text and code use transformers; images and audio typically use diffusion models.
- → The model optimises for plausible continuations, not verified ones.
- → Something outside the model must check the output — a human in generative use, machinery in agentic use.

One step of generation

The model's actual output is the distribution, not the word. Sampling turns it into a token, and the token is appended and fed back. Temperature controls how much probability mass outside the top candidate gets a chance — which is also why the same prompt can produce different results.

```
prompt = "The capital of France is"

# the model produces a distribution over the whole vocabulary
logits = model(prompt)              # shape: [vocab_size]
probs  = softmax(logits / temperature)

#   " Paris"   0.94
#   " located" 0.02
#   " a"       0.01
#   ...

next_token = sample(probs)          # append, then repeat with the longer prompt
```

03 The agent loop: perceive, decide, act, learn

1:22

> Four stages, repeated. Everything that makes an agent an agent lives in this cycle rather than in the model.

KEY POINTS

- → The cycle is perceive → decide → act → learn, repeated until done.
- → Only 'decide' is the model; the other three stages are ordinary software.
- → Feeding action results back into context is what allows an agent to recover from a bad step.
- → The loop has no inherent stopping condition — step budgets and timeouts are your responsibility.
- → Most agent bugs live in tool definitions and error handling, not in model reasoning.

A working agent loop

This is what almost every agent framework wraps. Note the three defences that have nothing to do with AI: a hard step budget, tool errors fed back as observations rather than raised, and a definite outcome when the budget is exhausted.

```
def run_agent(goal, tools, max_steps=20):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user",   "content": goal},
    ]

    for step in range(max_steps):
        reply = model.chat(messages, tools=tools)      # decide

        if not reply.tool_calls:                       # the agent says it is done
            return reply.content

        messages.append(reply)

        for call in reply.tool_calls:                  # act
            try:
                result = tools[call.name](**call.arguments)
            except Exception as e:
                result = f"ERROR: {e}"                 # failures are observations,
                                                       # not crashes
            messages.append({                          # learn
                "role": "tool",
                "tool_call_id": call.id,
                "content": str(result),
            })

    raise StepBudgetExceeded(goal, messages)           # never loop forever
```

04 The shared substrate: an LLM as reasoning engine

2:03

Both approaches usually rest on the same model. In an agent, its job is narrower than

people assume: choose the next action, interpret the last result.

KEY POINTS

- → LLMs are the reasoning layer for agents and the generator for chatbots — the same component in two roles.
- → Diffusion models cover most image and audio generation; the agentic reasoning layer is an LLM.
- → Per turn, the model's job is to emit a tool call with arguments, or a final answer.
- → The model is stateless; the message history is the state and the application owns it.
- → Debug agents by inspecting the exact context at the failing turn, not by theorising about reasoning.

What the model sees on turn four

The model has no memory between turns. Everything it can use is in this list, which the application assembled. If a needed fact is not here, the model cannot use it — and that is a context construction bug, not a reasoning failure.

```
[
  {"role": "system",
   "content": "You are a shopping agent. Confirm price before purchase."},
  {"role": "user",   "content": "Buy the AB-1042 under 200 EUR"},

  {"role": "assistant", "tool_calls": [search_inventory(sku="AB-1042", region="EU")]},
  {"role": "tool",      "content": "in_stock: 3, price: 189.00 EUR, warehouse: NL"},

  {"role": "assistant", "tool_calls": [check_price_history(sku="AB-1042", days=30)]},
  {"role": "tool",      "content": "min: 175.00, max: 210.00, current: 189.00"},

  # turn 4: given all of the above, emit the next tool call or a final answer
]
```

05 Generative in practice: the human curates

3:28

Generative workflows produce possibilities. A person selects, corrects and directs —

and that review step is load-bearing.

KEY POINTS

- → Generative workflows pair machine output with human judgement on every result.
- → They suit low-stakes, subjective, fast-iteration tasks.
- → The human review step is a component of the system, not overhead to optimise away.
- → Removing the reviewer without adding verification turns a generative system into an unguarded agentic one.
- → Cost and latency are bounded and predictable: one call, one result.

Generate options, do not generate the answer

A small design choice with a large effect on generative features. Returning several candidates makes the human's job selection rather than correction, which is faster and produces better outcomes than asking for one output and editing it.

```
def thumbnail_concepts(title, n=5):
    return [
        model.generate(
            f"Propose a thumbnail concept for a video titled {title!r}. "
            f"One sentence. Be specific and visual.",
            temperature=0.9,          # high: we want spread, not consensus
        )
        for _ in range(n)
    ]

# the human picks one and refines it — selection beats correction
```

06 Agentic in practice: multi-step work under a single goal

4:10

Agents earn their complexity on tasks that need ongoing management across many

steps, where waiting for a human between each one defeats the purpose.

KEY POINTS

- → Agents fit multi-step tasks where later steps depend on earlier results.
- → They fit work that unfolds over time and reacts to a changing environment.
- → If a task is one step, an agent is a slow, unreliable function call.
- → If every step needs approval, a form is a better interface than an agent.
- → Irreversible actions need limits enforced in code, never merely requested in the prompt.

Enforce limits outside the model

The prompt says 'under 200 EUR'. That is a request. This is a guarantee. Any action with real consequences should pass through a check the model cannot talk its way past, because the model is a probabilistic system and your budget is not.

```
def purchase(sku, price_eur, *, budget_eur, confirmed):
    if price_eur > budget_eur:
        return f"REFUSED: {price_eur} exceeds budget {budget_eur}"
    if not confirmed:
        raise NeedsHumanApproval(sku=sku, price=price_eur)
    return checkout(sku)

# the model can request a purchase; only this function can make one
```

07 Chain-of-thought: the internal dialogue before the action

5:38

The agent breaks a complex goal into smaller logical steps by generating its own

reasoning, then acts on the result.

KEY POINTS

- → Chain-of-thought decomposes a goal into ordered, executable steps.
- → The reasoning is generated text, so it can be logged, inspected and asserted on.
- → Generation is the cognitive engine: the same mechanism, aimed at planning instead of prose.
- → Stated reasoning is not a guaranteed explanation — a bad action can arrive with a convincing rationale.
- → Reasoning tokens cost latency and money on every turn; depth is a budget decision.

Decomposition producing a checkable plan

Asking for the plan as structured output rather than prose makes it a data structure you can validate before any action runs — checking that dependencies are acyclic, that no step calls an unavailable tool, and that the cost is within budget.

```
PLAN_SCHEMA = {
  "steps": [
    {"id": 1, "action": "gather_requirements",
     "needs": [],     "produces": "size, duration, budget"},
    {"id": 2, "action": "search_venues",
     "needs": [1],    "produces": "candidate venues"},
    {"id": 3, "action": "check_availability",
     "needs": [2],    "produces": "available venues"},
    {"id": 4, "action": "request_quotes",
     "needs": [3],    "produces": "pricing"}
  ]
}

# validate before executing anything
assert is_acyclic(plan)
assert all(s["action"] in TOOLS for s in plan["steps"])
```

08 Systems that know when to generate and when to act

6:20

The strongest systems are neither purely generative nor purely agentic — they explore

options through generation and commit to them through action.

KEY POINTS

- → Mature systems generate to explore and act to commit, switching deliberately.
- → Divide by reversibility: generate freely, act under a gate.
- → Good agents generate many candidates and execute few.
- → Generative evaluation judges the output; agentic evaluation judges goal completion, cost and side effects.
- → Building only output-quality evaluation leaves the failure path untested.

Generate three, verify, act on one

The generative and agentic halves in a single function. Candidates are free, so produce several. Execution is not, so only a validated candidate reaches it — and if none validate, escalating beats acting on the least bad option.

```
def act_on_best(goal, state, k=3):
    candidates = [propose_action(goal, state) for _ in range(k)]   # generative

    for action in rank_by_confidence(candidates):
        ok, reason = validate(action, state)        # schema, budget, permissions
        if ok:
            return execute(action)                                  # agentic
        log.info("rejected %s: %s", action, reason)

    raise NoValidAction(goal, candidates)           # escalate, do not improvise
```

Glossary

Generative AI

A system that produces content in response to a prompt and then stops. Reactive by construction: one request, one result.

Agentic AI

A system that pursues a goal across multiple self-directed actions, observing results and adjusting, with minimal human intervention.

Agent loop

The repeating cycle of perceive, decide, act and learn that defines an agent. The model supplies the decide step; the rest is ordinary software.

Tool (function calling)

A function exposed to the model with a name, description and parameter schema. Tools are the only way an agent can observe or change anything outside its context.

Chain-of-thought reasoning

Generating intermediate reasoning steps before answering or acting, so a complex goal is broken into smaller logical pieces.

Next-token prediction

The mechanism behind text generation: produce a probability distribution over the vocabulary and sample the next token, repeatedly.

Diffusion model

The architecture behind most image and audio generation, which produces output by iteratively denoising random noise.

Step budget

A hard cap on how many iterations an agent may run. Necessary because nothing in the model guarantees the loop terminates.

Message history

The accumulated record of instructions, model replies and tool results. Since the model is stateless, this list is the agent's entire memory.

Human in the loop

A required approval point before a consequential action proceeds. In generative workflows it is continuous; in agentic ones it is placed at specific gates.

Reason–act trace

An execution log alternating stated reasoning with the action taken, used to make an agent's behaviour inspectable after the fact.

Temperature

A sampling parameter controlling how much probability mass outside the top candidate is used. Higher values give more varied output.

Check yourself

Answer first, then open each one.

Two systems use the identical model, the identical prompt and the identical API. One is generative and one is agentic. What is actually different?

The control flow around the call. The agentic system puts the model inside a loop with tools, feeding each action's result back into context so the next turn can use it. Nothing about the model changed — which is why 'is this agentic?' is a question about your code, not about which model you bought.

An agent takes a nonsensical action at step 7. Why is 'the model reasoned badly' usually the wrong first hypothesis?

Because the model only sees the context you assembled for that turn. Far more often the tool description was ambiguous, a previous tool returned something unparseable, or the relevant fact was crowded out of a long context. Dump the message list at step 7 and the cause is usually visible immediately.

Your prompt instructs the agent never to spend more than 200 EUR. Why is that insufficient, and what replaces it?

A prompt is a request to a probabilistic system, so compliance is likely but not guaranteed, and any instruction can be undermined by unusual input or a long context. The limit has to be a check inside the purchase function, which refuses regardless of what the model asked for. Enforce consequential constraints in code; use the prompt to explain them.

Why does an agent tolerate a wrong step much better than a single generative call tolerates a wrong answer?

The agent observes the consequence — a failing test, a 404, an empty result — and gets another turn with that observation in context, so it can correct. A generative call produces its output and stops, with no opportunity to notice anything is wrong. Per-step reliability matters less when the loop can see and recover.

A task is a single well-defined step that a human must approve anyway. Why is an agent the wrong architecture?

The loop buys recovery across dependent steps and autonomy between them, and neither applies here. You pay unpredictable latency, unpredictable cost and non-determinism for benefits the task cannot use. A single call behind a form is faster, cheaper and testable.

Why is generating several candidate actions and executing one a better pattern than generating one action and executing it?

Generation is cheap and reversible while execution is expensive and often not, so the asymmetry favours producing spare candidates and spending the validation effort before committing. It also gives you a fallback: if the top candidate fails validation, the next is already available without another round trip.

Where to go next

- → Implement the twenty-line agent loop above against two real tools and deliberately make one of them fail intermittently — then watch whether your error handling actually lets the agent recover or just loops.
- → Take an agent you already have and log the full message list at every turn, then read the transcript of a run that went wrong; note how many failures are context construction rather than reasoning.
- → Measure the cost distribution of an agentic feature across 100 runs: tokens and wall time per run. The spread, not the mean, is what breaks capacity planning.
- → Build the no-progress detector and instrument how often it fires in normal operation — the number is usually higher than teams expect.
- → Take a task you currently solve with an agent, rewrite it as a fixed pipeline with one model call per stage, and compare reliability, cost and latency. Many agentic features are pipelines that were never tested as pipelines.

→ Read the ReAct paper (Yao et al., 2022) for the original formulation of interleaving reasoning traces with actions, which is the pattern nearly every agent framework now implements.
